

Carry-Lookahead Adder (CLA)

A **carry-lookahead adder** (CLA) or fast adder is a type of [electronics adder](#) used in digital logic. A carry-lookahead adder improves speed by reducing the amount of time required to determine carry bits. It can be contrasted with the simpler, but usually slower, [ripple-carry adder](#) (RCA), for which the carry bit is calculated alongside the sum bit, and each stage must wait until the previous carry bit has been calculated to begin calculating its own sum bit and carry bit. The carry-lookahead adder calculates one or more carry bits before the sum, which reduces the wait time to calculate the result of the larger-value bits of the adder. The [Kogge–Stone adder](#) (KSA) and [Brent–Kung adder](#) (BKA) are examples of this type of adder.

A ripple-carry adder works in the same way as pencil-and-paper methods of addition. Starting at the rightmost ([least significant](#)) digit position, the two corresponding digits are added and a result is obtained. It is also possible that there may be a carry out of this digit position (for example, in pencil-and-paper methods, "9 + 5 = 4, carry 1"). Accordingly, all digit positions other than the rightmost one need to take into account the possibility of having to add an extra 1 from a carry that has yet to come in from the next position to the right.

This means that no digit position can have an absolutely final value until it has been established whether or not a carry is coming in from the right. Moreover, if the sum without a carry is 9 (in pencil-and-paper methods) or 1 (in binary arithmetic), it is not even possible to tell whether or not a given digit position is going to pass on a carry to the position on its left. At worst, when a whole sequence of sums comes to ...99999999... (in decimal) or ...11111111... (in binary), nothing can be deduced at all until the value of the carry coming in from the right is known, and that carry is then propagated to the left, one step at a time, as each digit position evaluated "9 + 1 = 0, carry 1" or "1 + 1 = 0, carry 1". It is the "rippling" of the carry from right to left that gives a ripple-carry adder its name, and its slowness. When adding 32-bit integers, for instance, allowance has to be made for the possibility that a carry could have to ripple through every one of the 32 one-bit adders.

Carry-lookahead depends on two things:

1. Calculating for each digit position whether that position is going to propagate a carry if one comes in from the right.
2. Combining these calculated values to be able to deduce quickly whether, for each group of digits, that group is going to propagate a carry that comes in from the right.

Supposing that groups of four digits are chosen the sequence of events goes something like this:

1. All 1-bit adders calculate their results. Simultaneously, the lookahead units perform their calculations.
2. Assuming that a carry arises in a particular group, that carry will emerge at the left-hand end of the group within at most five gate delays and start propagating through the group to its left.
3. If that carry is going to propagate all the way through the next group, the lookahead unit will already have deduced this. Accordingly, *before the carry emerges from the next group*, the lookahead unit is immediately (within one gate delay) able to tell the *next* group to the left that it is going to receive a carry – and, at the same time, to tell the next lookahead unit to the left that a carry is on its way.

Generate (G_i) and propagate (P_i) signals for each column:

A column will generate a carry out if A_i AND B_i are both 1.

$$G_i = A_i B_i$$

A column will propagate a carry in to the carry out if A_i OR B_i is 1.

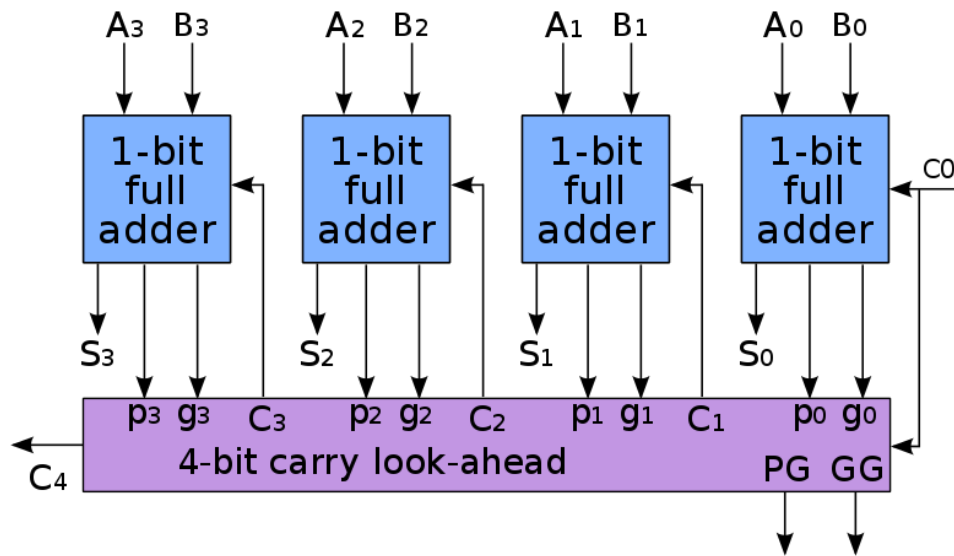
$$P_i = A_i + B_i$$

The carry out of a column (C_i) is:

$$C_{i+1} = A_i B_i + (A_i + B_i)C_i = G_i + P_i C_i$$

Where,

$$\begin{aligned}
C_1 &= a_0b_0 + (a_0+b_0)c_0 = g_0 + p_0c_0 \\
C_2 &= a_1b_1 + (a_1+b_1)c_1 = g_1 + p_1c_1 = g_1 + p_1g_0 + p_1p_0c_0 \\
C_3 &= a_2b_2 + (a_2+b_2)c_2 = g_2 + p_2c_2 = g_2 + p_2g_1 + p_2p_1g_0 + p_2p_1p_0c_0 \\
C_4 &= a_3b_3 + (a_3+b_3)c_3 = g_3 + p_3c_3 = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0 + p_3p_2p_1p_0c_0 \\
q_i &= a_ib_i \quad p_i = a_i + b_i
\end{aligned}$$



- Step 1: compute *generate* (G) and *propagate* (P) signals for columns (single bits)
- Step 2: compute G and P for k -bit blocks
- Step 3: C_{in} propagates through each k -bit propagate/generate block
- Delay of an N -bit carry-lookahead adder with k -bit blocks:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

where

- t_{pg} : delay of the column generate and propagate gates
- t_{pg_block} : delay of the block generate and propagate gates
- t_{AND_OR} : delay from C_{in} to C_{out} of the final AND/OR gate in the k -bit CLA block

An N -bit carry-lookahead adder is generally much faster than a ripple-carry adder for $N > 16$

- Computes the carry in (C_{i-1}) for each of the columns as fast as possible and then computes the sum:

$$S_i = (A_i \hat{\wedge} B_i) \hat{\wedge} C_i$$

- Computes G and P for 1-bit, then 2-bit blocks, then 4-bit blocks, then 8-bit blocks, etc. until the carry in (generate signal) is known for each column has $\log_2 N$ stages
- A carry in is produced by being either *generated* in a column or *propagated* from a previous column.
- Define column -1 to hold C_{in} , so $G_{-1} = C_{in}$, $P_{-1} = 0$
- Then, the carry in to col. i = the carry out of col. $i-1$:

$$C_{i-1} = G_{i-1:-1}$$

$G_{i-1:-1}$ is the generate signal spanning columns $i-1$ to -1 .

There will be a carry out of column $i-1$ (C_{i-1}) if the block spanning columns $i-1$ through -1 generates a carry.

- Thus, we rewrite the sum equation: $S_i = (A_i \hat{\wedge} B_i) \hat{\wedge} G_{i-1:-1}$

Goal: Compute $G_{0:-1}$, $G_{1:-1}$, $G_{2:-1}$, $G_{3:-1}$, $G_{4:-1}$, $G_{5:-1}$, ... (These are called the *prefixes*)

4-bit Carry-Lookahead Adder (CLA):- Code using Data Flow Modeling in Verilog HDL.

```
module carry_look_ahead_adder (a, b, cin, s, cout);

    input [3:0] a, b;
    output [3:0] s;
    input cin;
    output cout;

    wire [3:1] c; //carry between full adder block
    wire [3:0] g, p; //generation and propogation port

    assign g[0] = a[0] && b[0]; // g(i) = a(i) . b(i)
    assign g[1] = a[1] && b[1];
    assign g[2] = a[2] && b[2];
    assign g[3] = a[3] && b[3];

    assign p[0] = a[0] ^ b[0]; // p(i) = a(i) Exor b(i)
    assign p[1] = a[1] ^ b[1];
    assign p[2] = a[2] ^ b[2];
    assign p[3] = a[3] ^ b[3];

    assign c[1] = g[0] || (p[0] && cin); //c1 = g0 + (p0 . c0)
    assign c[2] = g[1] || (p[1] && g[0]) || (p[1] && p[0] && cin); //c2 = g1 + (p1 . g0) + (p1 . p0. cin)
    assign c[3] = g[2] || (p[2] && g[1]) || (p[2] && p[1] && g[0]) || (p[2] && p[1] && p[0] && cin); //c3
    = g2 + (p2 . g1) + (p2 . p1. g0) + (p2 . p1 . p0 . co)

    assign cout = g[3] || (p[3] && g[2]) || (p[3] && p[2] && g[1]) || (p[3] && p[2] && p[1] && g[0]) ||
    (p[3] && p[2] && p[1] && p[0] && cin);

    assign s[0] = p[0] ^ cin; //s(i) = p(i) Exor c(i)
    assign s[1] = p[1] ^ c[1];
    assign s[2] = p[2] ^ c[2];
    assign s[3] = p[3] ^ c[3];

endmodule
```